

# AI 编程与软件的未来

从页面云到智能体基础设施

Stanford MS&E 435: Economics of the AI Supercycle

五道口纳什 & Codex

2026 年 8 月 28 日



视频频道: Stanford Online

主持与嘉宾: Apoorv Agrawal; 嘉宾 Guillermo Rauch (Vercel)

课程日期: 2026 年 5 月 22 日 上传日期: 2026 年 6 月 23 日

视频时长: 49:22

视频链接: <https://www.youtube.com/watch?v=HA7lZd7zk3M>

配套材料: [Stanford MS&E 435 Course Materials](#)

笔记证据: 1,091 条人工英文 CC、1920×1080 原视频、Week 8 三项延伸材料

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

# 目录

|                                         |           |
|-----------------------------------------|-----------|
| 阅读说明                                    | 3         |
| <b>1 从个人经历到软件民主化</b>                    | <b>3</b>  |
| 1.1 开放工具与商业基础设施为何能够共存                   | 3         |
| 1.2 部署平台隐藏的复杂性                          | 3         |
| 1.3 编程史可以被理解为访问权扩张史                     | 4         |
| 1.4 本章小结                                | 4         |
| <b>2 写代码不等于交付：部署才启动学习</b>               | <b>4</b>  |
| 2.1 编码智能体与部署平台的互补关系                     | 4         |
| 2.2 云的部署对象从 Page 扩展到 Agent              | 5         |
| 2.3 从毫秒请求到全天任务                          | 5         |
| 2.4 本章小结                                | 6         |
| <b>3 从 Pixels 到 Tokens：智能体基础设施的总体框架</b> | <b>6</b>  |
| 3.1 从人写代码的弹性计算到智能体写代码                   | 6         |
| 3.2 人类界面的回归：更丰富，而非更贫瘠                   | 7         |
| 3.3 智能体基础设施的三个面                         | 7         |
| 3.4 模型的技术先验与「积木经济」                      | 7         |
| 3.5 拓展阅读                                | 8         |
| 3.6 本章小结                                | 8         |
| <b>4 智能体云的核心原语：网关、沙箱、安全与自动运维</b>        | <b>9</b>  |
| 4.1 软件从知识入口变成任务参与者                      | 9         |
| 4.2 AI Gateway：把 token 调用纳入生产控制面        | 9         |
| 4.3 Sandbox：既是智能体的计算机，也是风险边界            | 10        |
| 4.4 自动驾驶云：感知、变更、验证与护栏                   | 10        |
| 4.5 Meta 与 Notion：供应商演讲中的两条采用路线         | 11        |
| 4.6 本章小结                                | 12        |
| <b>5 SaaS 的分层可塑化：呈现层可生成，系统记录继续复用</b>    | <b>12</b> |
| 5.1 「软件会死亡」为什么是错误的问题                    | 12        |
| 5.2 停车管理与销售工作台：两个严格限定的案例                | 13        |
| 5.3 什么会被抛弃，什么仍值得复用                      | 13        |
| 5.4 本章小结                                | 14        |

|                                             |           |
|---------------------------------------------|-----------|
| <b>6 一次性软件与新的开发、支持闭环</b>                    | <b>14</b> |
| 6.1 一次客户沟通，也可以是一份软件的完整生命周期                  | 14        |
| 6.2 「软件基本免费」只描述边际生成环节                       | 14        |
| 6.3 AI 的反身性：可生成的认知会增加生成频率                   | 14        |
| 6.4 新支持对象：客户智能体带着轨迹来到平台                     | 15        |
| 6.5 本章小结                                    | 15        |
| <b>7 局部推理、可组合性与全栈复用</b>                     | <b>15</b> |
| 7.1 高选择率不是单因果故事                             | 15        |
| 7.2 Tailwind 的局部推理：牺牲整洁感，换取边界               | 16        |
| 7.3 有限上下文为何使可组合性成为规模化前提                     | 16        |
| 7.4 全栈策略：共同底座、dogfooding 与真实负载              | 16        |
| 7.5 本章小结                                    | 17        |
| <b>8 价值归属、开放商业与「token 速度」</b>               | <b>17</b> |
| 8.1 价值检验：模型能力能否抵达有用产品                       | 17        |
| 8.2 域名、DNS 与工程师仍然承担的责任                      | 18        |
| 8.3 承压类别与受益接口                               | 18        |
| 8.4 为什么「No more rate limits」只是挑衅            | 19        |
| 8.5 本章小结                                    | 19        |
| <b>9 总结与延伸：可靠性、能源与未决问题</b>                  | <b>19</b> |
| 9.1 多星球兴趣背后的可靠性直觉                           | 19        |
| 9.2 「Energy goes in, intelligence goes out」 | 20        |
| 9.3 全讲的四条教学主线                               | 20        |
| 9.4 仍待解决的问题                                 | 21        |
| 9.5 延伸材料的正确使用方式                             | 21        |
| 9.6 本章小结                                    | 21        |

## 阅读说明

这份笔记以 1,091 条 YouTube 人工英文 CC、49:22 的 1080p 课堂录像和 Stanford MS&E 435 Week 8 材料页为来源。原视频未提供章节，本文依据完整字幕的话题转折重建教学顺序。课堂中出现的命令、代码和产品操作只作分析，没有被执行。

本文严格区分三类语句：主持人的开场介绍或追问、Guillermo Rauch 的经验判断与 Vercel 公司自述、以及笔记为帮助理解而做的工程性展开。演讲中的精确比例、增长数字和客户案例均保留其当时口径，不写成已经独立审计的行业事实。

### 本文的核心问题

当生成代码越来越容易，软件经济中真正稀缺的东西会迁移到哪里？本讲的答案逐步指向：可运行的部署、长时任务的状态与可观测性、安全沙箱、可组合接口、可靠运维，以及支撑这些能力的能源。

## 1 从个人经历到软件民主化

### 1.1 开放工具与商业基础设施为何能够共存

主持人用一组简短履历介绍 Rauch：他成长于布宜诺斯艾利斯郊区，小时候自学编程，并因为当时西班牙语软件手册很少而借助英文手册学习英语。对本讲而言，这段经历的意义不是营造「辍学即成功」的传记，而是交代一个持续影响其产品观的约束：文档、语言、免费工具与地理位置，都可能决定一个人能否进入软件世界。 [00:00:36--00:01:43]

Rauch 将这种经验转成了产品原则。Vercel 构建了大量开源工具，再在开放工具之上提供部署、安全和扩缩容服务。这条路径包含一个可复用的逻辑：开源降低学习与创作门槛；当作品真正面向全球用户时，可靠运行的复杂性仍然存在，专业平台因此获得商业价值。 [00:02:11--00:03:54]

### 证据边界

开场中的 Next.js 排名、Vercel 估值、早期工作年龄与客户关系，均是主持人或 Rauch 的现场表述。它们可以解释人物动机，但不是本文独立核实的传记或公司尽调结论。

### 1.2 部署平台隐藏的复杂性

AWS、Google Cloud 和 Azure 早已能部署网站，但 Rauch 回忆，即使他已有多年工程经验，将新项目部署上线仍花了数周。先进技术并不自动带来良好的开发者体验：前端开发者想把用户界面交付给全球用户，却必须先理解容器、网络、负载均衡和分布式运维。 [00:03:54--00:04:41]

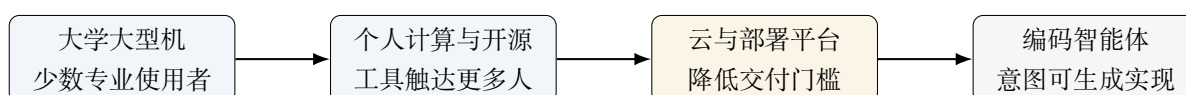
Vercel 的早期承诺是改写这个任务边界：开发者专注于面向人的应用，平台负责部署、低延迟、扩缩容和负载均衡。底层能力并未消失，它们被封装到更窄、更易用的接口后面，从而扩大了能够交付软件的人群。 [00:04:41--00:06:10]

### 1.3 编程史可以被理解为访问权扩张史

Rauch 把编码智能体放到一条更长的轨迹上：早期只有大学大型机使用者能编写软件，后来个人计算、开源工具与云平台不断扩大参与者群体，AI 又让不会手写全部代码的人能够表达意图并生成实现。 [00:06:10--00:06:50]

#### 三次门槛下移

开源降低「获得工具」的门槛，部署平台降低「交付软件」的门槛，编码智能体则降低「把意图图变成实现」的门槛。每次扩张都会把稀缺性推向下一层尚未充分抽象的基础设施。



每次抽象都扩大参与者范围，同时把稀缺性推向下一层：运行、权限、可靠性与治理。

图 1: 软件创作访问权的扩张路径（讲义重绘；依据 00:06:10–00:06:50）

### 1.4 本章小结

软件民主化不只意味着更多人能写代码，还意味着每一层抽象都会重新分配责任。开放工具、简化部署和自然语言编程共同扩大了软件供给；下一个问题于是变成：代码生成之后，谁负责让它真正运行？

## 2 写代码不等于交付：部署才启动学习

### 2.1 编码智能体与部署平台的互补关系

Rauch 把编码智能体与部署基础设施比作「花生酱与果酱」。这个略带营销色彩的比喻准确分开了两种能力：前者生成、修改和组织实现，后者把实现放进用户可访问的运行环境。 [00:06:50--00:07:19]

#### 部署才让学习开始（00:07:19–00:07:35）

**Rauch (意译):** 仅仅写出代码并不构成交付；当运行中的版本真正出现在客户面前，学习才开始。

这里的「学习」是一个完整的产品回路：用户面对可运行版本，产生行为、错误、转化或满意度信号，团队再用这些证据修改实现。GitHub、SourceForge 和 Freshmeat 被用作反例：人类很擅长保存源码，但代码库与可运行服务之间还隔着依赖、构建、配置、密钥、数据、网络和持续运维。

[00:07:35--00:08:06]

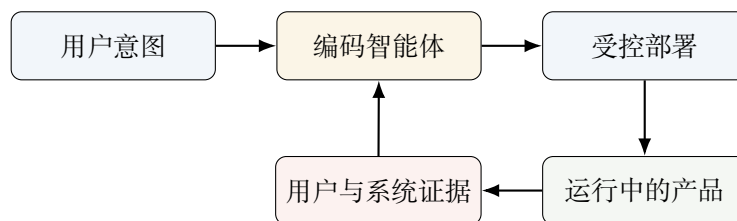


图 2: 从生成实现到真实学习的交付闭环（讲义重绘）

## 2.2 云的部署对象从 Page 扩展到 Agent

如果今天重新创建 AWS，Rauch 设想它也许会更像「Amazon Agent Services」。这是一个反事实命名，不是 Amazon 的真实产品公告。它的作用是强迫我们重新定义云的基本对象：页面与短时服务通常快速返回内容；智能体却可能持续推理、调用工具、保存中间状态，并等待外部结果。

[00:08:27--00:09:13]

|           | Traditional Cloud                          | Agent Cloud                                      |
|-----------|--------------------------------------------|--------------------------------------------------|
| UI        | Static & semi-static<br>ONE SHOT RESPONSES | Full dynamic & generative<br>STREAMING RESPONSES |
| FRAMEWORK | UI & pages<br>REACT / NEXT.JS              | AI & agents<br>AI SDK                            |
| CDN       | Pixels<br>CLOUDFRONT                       | Tokens<br>AI GATEWAY                             |
| COMPUTE   | Human-written code<br>EC2                  | Agent-written code<br>SANDBOX                    |
|           | Foreground<br>Quick / CPU-bound<br>LAMBDA  | Background<br>Long-running / IO-bound<br>FLUID   |

图 3: 传统云与智能体云的对象、框架、分发和计算原语对照<sup>1</sup>

<sup>1</sup> 视频实际帧 00:08:40；对应人工 CC 00:08:27-00:09:13；1920×1080 原始画面；未裁剪、未增强

## 2.3 从毫秒请求到全天任务

传统网页工程把响应时间压到毫秒级，因为用户正在等待页面。Rauch 援引了一个经典行业数字：Amazon 页面每慢 100 ms，转化率下降 1%。本任务没有追溯原始实验、时间、样本和因果口径，所以该数字只能标为「Rauch 在演讲中引用」。[00:09:28--00:09:47]

智能体任务则把时间尺度逐级拉长：数秒、数分钟、数小时，直到运行一整天后才返回报告、分析或软件。任务更慢不必然代表体验更差；若它包含调查、规划、工具调用、构建与验证，完成更多工作可能比立即返回浅层答案更有价值。[00:09:47--00:10:29]

### 时间尺度会重写基础设施

长时任务需要可恢复会话、中间状态、异步通信、检查点、成本追踪与进度可观测性。运行越久，遇到网络错误、限额、权限变化和依赖失效的概率越高，恢复与治理因此成为一等能力。

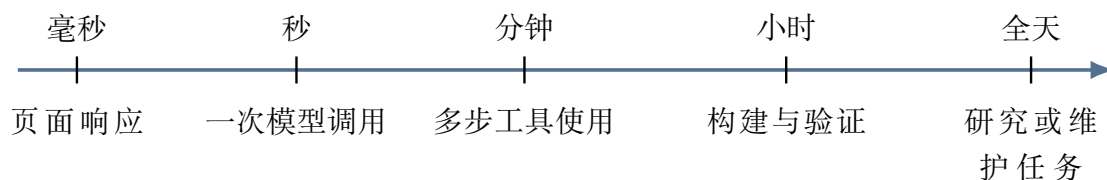


图 4: 任务时间尺度扩大后，检查点、恢复、进度与成本控制逐渐成为一等能力（讲义重绘；依据 00:09:47–00:10:29）

## 2.4 本章小结

源码库只是可交付软件的中间态。当编码智能体把生成与部署更紧密地连在一起，平台同时要负责权限、审查、状态、恢复和可观测性。「Page 到 Agent」的真正变化，是云开始承载长时、有状态且能调用工具的运行对象。

## 3 从 Pixels 到 Tokens：智能体基础设施的总体框架

### 3.1 从人写代码的弹性计算到智能体写代码

EC2 把「按需取得计算实例」商品化。在 Rauch 的经济化解释中，过去的云需求受到人类程序员数量约束：先有人写出程序，程序才会消耗更多计算。编码智能体却能生成、修复和保护软件，攻击与防御两侧也都需要计算。他用  $10 \times / 20 \times / 100 \times$  描述潜在创作者扩张；这是无时限、无样本的范围性判断，不是可直接绘制为预测曲线的数据。 [00:10:50--00:11:54]

Rauch 随后把云的输出对象概括为「pixels 扩展到 tokens」。过去平台主要把页面、界面与应用呈现给人；现在它还在以 token 形式传递模型输出。这使按席位定价与按 token / 用量定价的边界变得模糊：一个人可以指挥智能体执行大量任务，席位不再稳定代表资源消耗。但 token 只是用量单位，并不直接测量任务质量、商业价值或人类意义上的智能。 [00:11:54--00:12:29]

### Tokens 与 Pixels 不是零和替代

字幕在 00:12:29 开始一个关键限定：Rauch 随即表示他仍然看好以人为中心的体验。因此，token 是新增的工作负载与计量方式，页面、像素和人类界面仍负责呈现、解释和判断结果。

图 3 把这种变化放在同一技术栈内：像素分发没有消失，token 分发、沙箱和长时计算作为新原语被加到旁边。图中的产品映射是 Vercel 演示口径，不是云行业的唯一分类法。



### 3.2 人类界面的回归：更丰富，而非更贫瘠

智能体在后台思考、调用工具和生成结果，人仍需要理解结果、比较选项并判断后果。Rauch 以 Microsoft Encarta 与 Wikipedia 为个人化对比：前者留给他的印象是互动、会「活起来」的百科体验，后者则常以大段文字与少量图片呈现。随着即时生成视频、3D 模型与沉浸式界面的成本下降，他预测更「异想天开」的网络体验会回归。这是趣味与未来判断，并非行业已完成的状态。

[00:12:31--00:13:47]

### 3.3 智能体基础设施的三个面

如果只把智能体基础设施理解为「把编码智能体生成的网站部署出去」，就会遗漏两类同样重要的需求。Rauch 用一个三角形归纳三组关系： [00:13:47--00:14:48]

1. **给编码智能体运行**：为 Claude Code、Codex、v0 等生成的软件提供部署与运行环境。
2. **用基础设施构建智能体**：企业面向学生、客户或员工交付的核心软件，可能从一组页面变为能持续工作的智能体。
3. **让智能体运行基础设施**：智能体参与配置、监控、优化和验证生产系统。

#### 三角形记忆法

给智能体运行、用平台造智能体、让智能体运行平台。三者共享底层计算，但它们的业务状态、权限模型和失败后果并不相同。

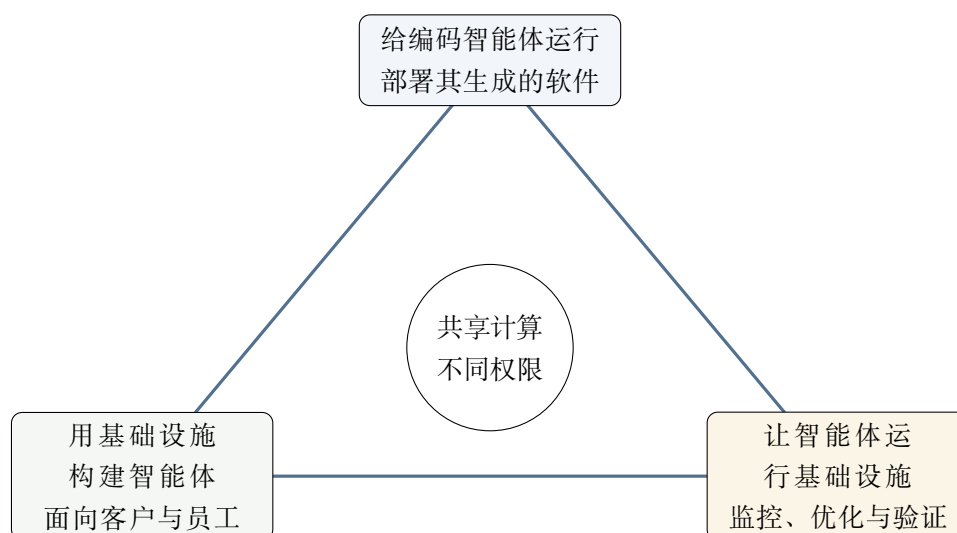


图 5: 智能体基础设施的三组关系（讲义重绘；依据 00:13:47–00:14:48）

### 3.4 模型的技术先验与「积木经济」

编码智能体并非在空白世界中做技术选型。Rauch 认为，模型从公开互联网中学会了 Next.js、React、shadcn 以及相关文档和开源代码，因此对某些技术拥有更充足的可用上下文。他还援引一份演讲所示报告，称 shadcn 在该测试中为 90.1%，Vercel 部署选择为 100%。字幕没有给出报告



标题、任务样本和统计口径，这些数字只能用来理解 Rauch 的论证，不能直接外推为市场份额。

[00:14:48--00:15:45]

智能体理论上可以为每个任务重写操作系统和网络栈，但 token、时间、上下文与执行成本会惩罚「为了造苹果而重新发明宇宙」。开放、稳定、可组合的构件为智能体提供了更便宜的路径。Rauch 借用 Mitchell Hashimoto 的说法将其称为 **积木经济 (block economy)**：竞争的一部分变成「谁能提供更容易被智能体发现、理解和复用的积木」。 [00:15:45--00:17:09]

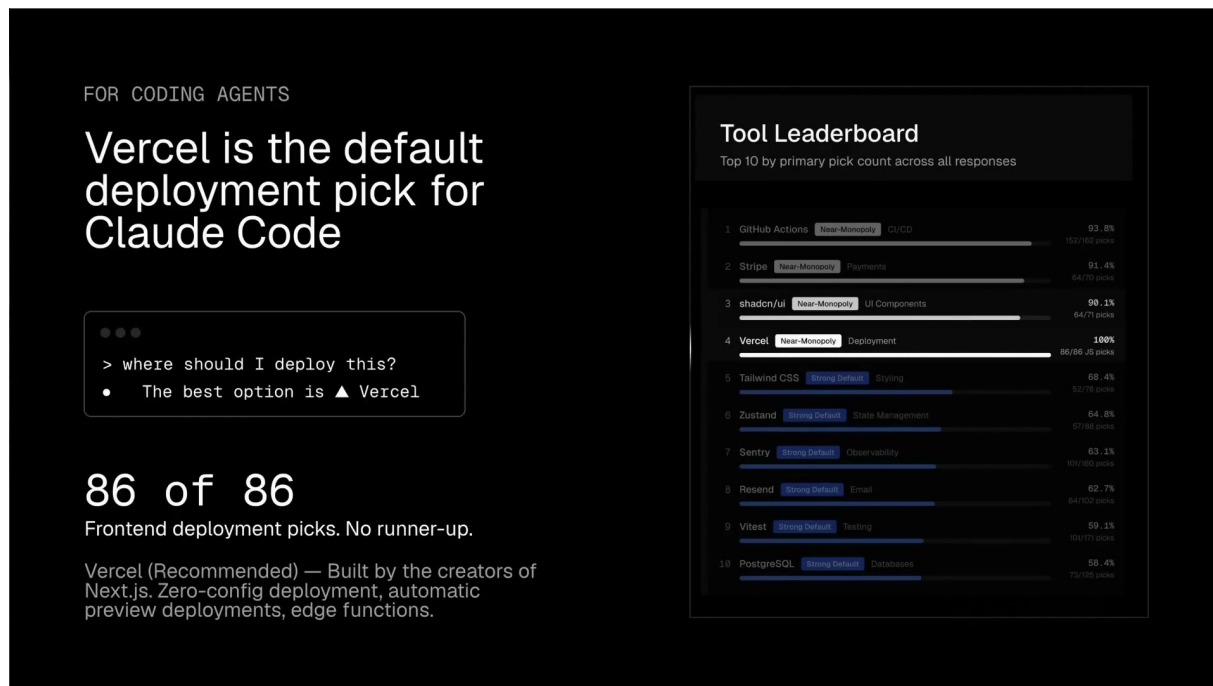


图 6: 演讲展示的 Claude Code 工具选择榜单；图中百分比只属于该页所示测试口径<sup>2</sup>

<sup>2</sup> 视频实际帧 00:15:26；对应人工 CC 00:15:21–00:15:45；1920×1080 原始画面；未裁剪、未增强

### 什么是智能体友好性？

这里的 agentic ergonomics 指产品是否容易被智能体发现、授权、调用和继续组合。公开文档、稳定 API、可理解错误、明确权限与结构化结果都会影响智能体能否经济地复用一项能力。这是对演讲机制的教学性展开，不是 Rauch 给出的正式规范。

## 3.5 拓展阅读

- Andrej Karpathy, *Software 2.0* (2017)：用数据、网络架构与训练后权重解释可执行表示的变化，可作为「表示变化会带动工具链变化」的历史背景。
- Stanford MS&E 435 [Week 8 材料页](#)：还列出 Karpathy 的 vibe coding 原帖与 Rauch 的一场延伸访谈。它们用于背景和对照，不替代本讲字幕证据。

## 3.6 本章小结

智能体时代同时扩大了计算需求与界面可能性。云平台开始流式传递 token，但人类仍需要丰富界面来理解智能体的结果。智能体基础设施同时支持三类关系，而开放且可组合的构件则会因节

省上下文、时间与重建成本，更容易进入智能体的选择集。

## 4 智能体云的核心原语：网关、沙箱、安全与自动运维

### 4.1 软件从知识入口变成任务参与者

Rauch 用 Vercel 的客服智能体解释「产品本身成为智能体」：据其公司自述，该智能体回答了 93% 的用户咨询，提高了支持效率，也让更多免费用户能够获得服务。[00:17:09--00:17:37] 这个数字描述的是回答占比；演讲没有给出完全解决率、转人工率、满意度、样本周期或评估方法。因此，不能把它改写成「93% 的问题已经解决」。

知识库要求用户自己搜索、阅读并执行步骤；任务型智能体则可以组织上下文、选择工具并推进工作。Vercel 在这里既是平台提供者，也是平台使用者：它用自己的构件与运行环境构建支持智能体。[00:17:38--00:17:54]

### 4.2 AI Gateway：把 token 调用纳入生产控制面

网页时代的 CDN 为页面和像素提供缓存、加速、规模化传递与安全边界。Rauch 把 AI Gateway 类比为「token 的 CDN」：应用不必把每次模型请求直接绑定到单一提供方，而是经过统一入口获得观测、故障切换、安全策略、缓存和负载均衡。[00:17:54--00:18:31]

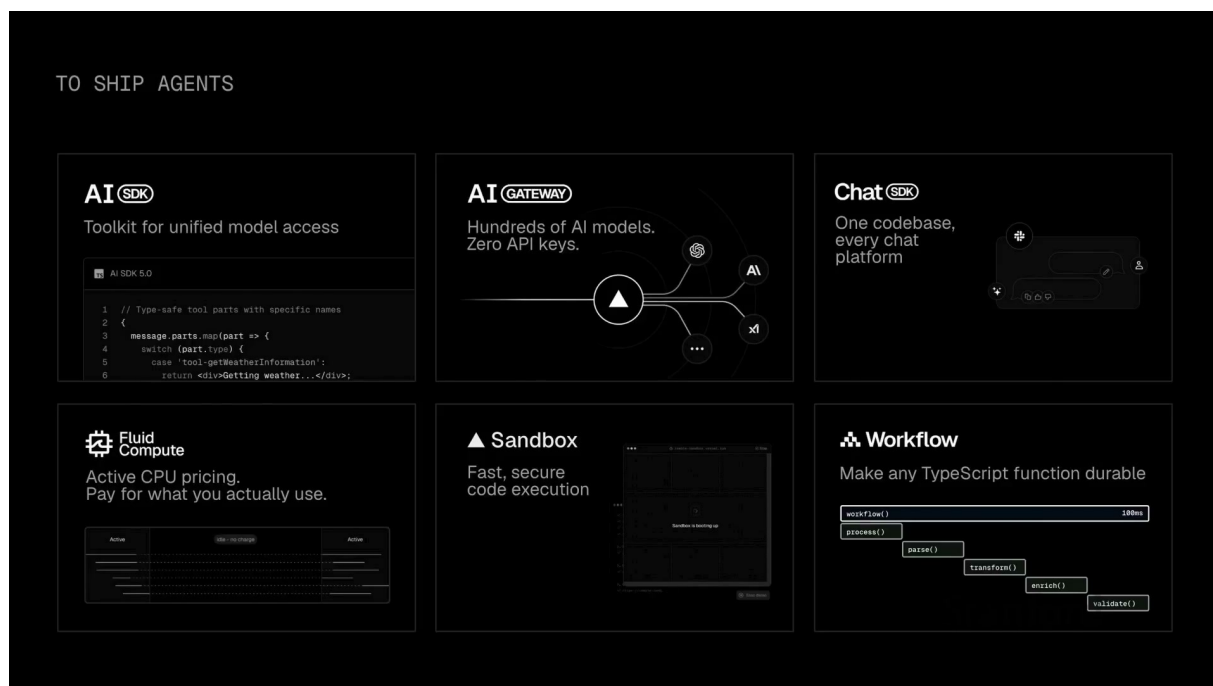


图 7: 演讲中的智能体交付工具栈：AI SDK、AI Gateway、Chat SDK、Fluid Compute、Sandbox 与 Workflow<sup>3</sup>

<sup>3</sup> 视频实际帧 00:18:24；对应人工 CC 00:17:54-00:19:17；1920×1080 原始画面；未裁剪、未增强

统一入口的经济价值在于匹配「请求难度」与「推理成本」。Rauch 用一个流行玩笑说明反例：用户只说一句 **thanks**，系统却可能唤起昂贵模型和「约 300 块 GPU」回答 **you're welcome**。网

关可以把简单意图交给较小模型，或命中语义缓存。[00:18:47--00:19:10] 这里的 300 GPU 是演讲中的夸张说法，不是本项目核验的推理资源测量。

#### 语义相似不等于结果可以安全复用

模型请求的等价性还受上下文、身份、权限、时效和副作用影响。网关可以提供缓存控制点，但不能仅凭两句话「看起来相似」就复用高风险结果。演讲没有给出正式路由或缓存算法，这里保留为工程边界。

### 4.3 Sandbox：既是智能体的计算机，也是风险边界

语言模型可以形成计划，但检查仓库、生成文件、调用工具与验证结果都需要执行环境。Rauch 把 Sandbox 类比为 EC2，并称其为智能体的基础计算单元；这个类比强调按需取得工作环境，不表示沙箱与通用虚拟机在实现上完全相同。[00:19:17--00:19:29]

他用知识工作者入职说明预配置的重要性：公司会发一台装好必要软件的电脑，平台也可以给智能体一台预配置、隔离且可销毁的「电脑」。Rauch 还认为模型在后训练中接触过容器式 playground，因此得到计算环境后能完成更多工作；本段没有给出模型范围或效果实验。[00:19:29--00:20:27]

| 角色   | 提供什么                | 仍需验证什么                 |
|------|---------------------|------------------------|
| 能力载体 | 文件系统、进程、依赖、工具与可观察输出 | 环境是否可复现，工具是否真的完成目标     |
| 风险边界 | 与宿主和生产系统隔离，任务结束后可销毁 | 最小权限、网络出口、凭据隔离、审计与生命周期 |

计算环境扩大智能体的行动能力，也同时扩大潜在影响面。Rauch 把它与个人电脑革命相类比：电脑带来生产力，也带来病毒、钓鱼和欺诈；智能体同样可能被诱导外传数据。因此，沙箱不是「安全已经完成」的证明，而是实施安全控制的边界。[00:20:27--00:21:02]

### 4.4 自动驾驶云：感知、变更、验证与护栏

传统运维仍让人在高压反馈回路中值守。Rauch 描述 pager/on-call 的夜间告警，并转述 Stripe 创始人与 PagerDuty 鸭叫铃声的轶事，用来表现长期运维压力；这是一则二手故事，不是医学证据。[00:21:02--00:21:48]

他设想未来的云能够自我配置、自我优化：智能体观察性能，形成变更，提交 PR 或在受控条件下部署，再以技术指标或客户转化率检验结果。[00:21:48--00:22:35] 其中「快两倍」只是愿景里的示例话术。真正可迁移的结构，是变更必须带证据并进入可回滚闭环。

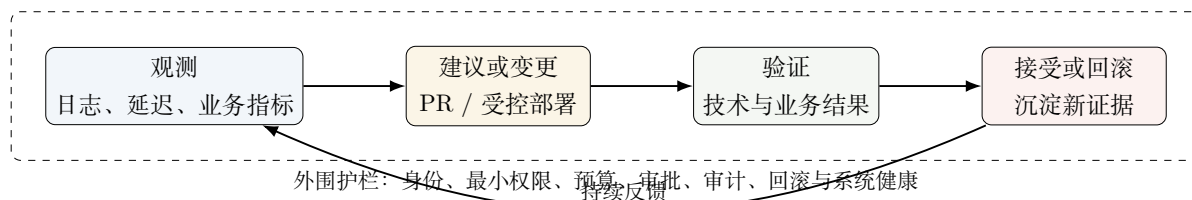


图 8: 自动驾驶云的目标闭环及不可省略的护栏（讲义重绘；依据 00:21:02–00:22:35, 并连接 00:47:05–00:47:40）

#### 自动运维的完成条件

产生一份修改建议不等于系统已经优化；部署成功也不等于业务更好。只有观察、变更、独立验证与安全回滚闭合起来，智能体才可能从建议者逐步变成受控执行者。

### 4.5 Meta 与 Notion：供应商演讲中的两条采用路线

Rauch 先承认 Meta 已有深厚的内部开发者基础设施，随后从 Vercel 视角称：编码智能体带来的快速迭代需求，让 Vercel 进入部分相关工作负载。他还提到 vibe-coded 内部工具、模型训练和 Meta.AI 交付，但没有提供项目基线、对照周期或可量化提速数据。 [00:22:35--00:23:20]

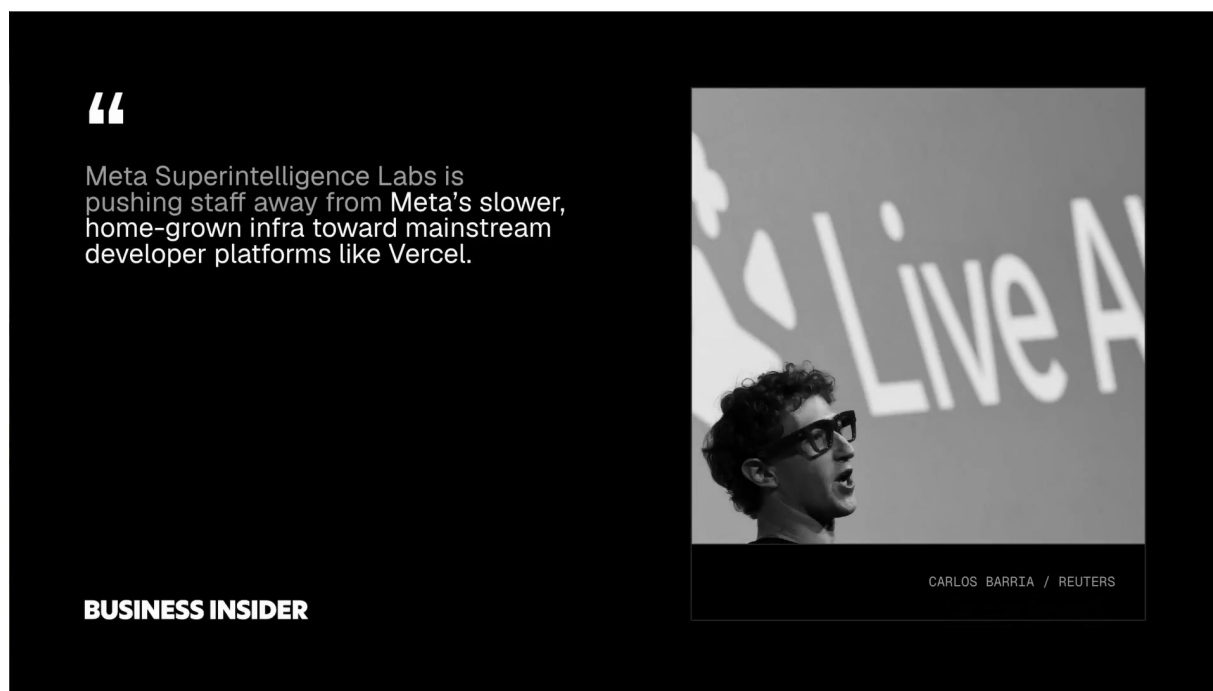


图 9: 演讲引用的 Business Insider 页面；该图保留供应商展示与媒体引文语境，不构成独立客户验收<sup>4</sup>

<sup>4</sup> 视频实际帧 00:22:42；对应人工 CC 00:22:35–00:23:20；1920×1080 原始画面；未裁剪、未增强

Notion 案例强调另一条路线：已有软件把产品能力逐步转向智能体形态。演讲引用页写的是 Notion 使用 Vercel 发布 Notion Workers，该平台让开发者构建和部署 AI agents；同一页面随后评价 Vercel 的工具同时适配人类开发者与 AI 智能体。Rauch 又提到课堂转录场景，但字幕并未完整界定每项能力的上线状态。[00:23:20--00:23:47]

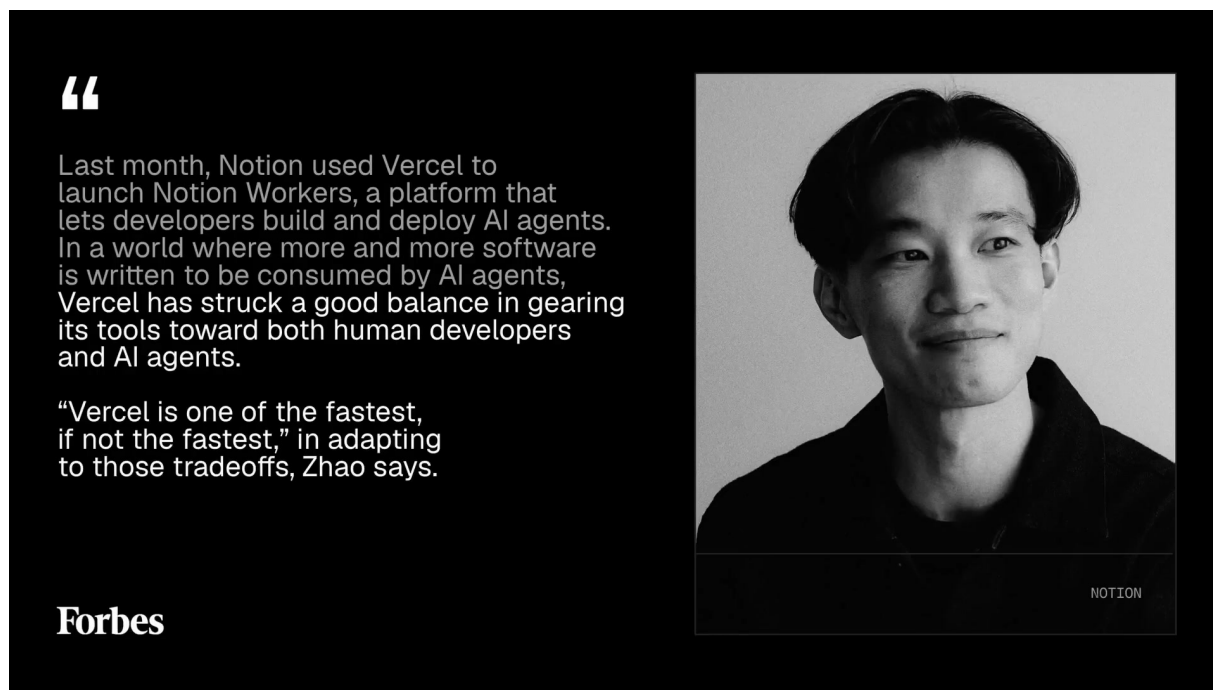


图 10: 演讲引用的 Forbes 页面：Notion 用 Vercel 发布 Notion Workers；页面另评价 Vercel 工具同时适配人类开发者与 AI 智能体<sup>5</sup>

<sup>5</sup> 视频实际帧 00:23:26；对应人工 CC 00:23:20-00:23:47；1920×1080 原始画面；未裁剪、未增强

Meta 展示的是「已有大规模平台的组织仍追求新的迭代速度」，Notion 展示的是「传统软件产品逐步加入智能体能力」。两者都来自 Vercel 创始人的客户案例讲述，证据强度低于客户公开技术报告。Rauch 随后用「AI 或智能体领域的 AWS」概括 Vercel 定位；这是一种公司战略叙事，不是中立行业认证。[00:23:47--00:24:01]

## 4.6 本章小结

智能体云不是一个模型 API，而是一组把模型能力变成可运行服务的基础原语：Gateway 管理 token 路径，Sandbox 提供受控执行环境，安全机制限制能力外溢，自动驾驶闭环用真实指标验证变更。客户案例说明这些原语可以进入内部工具或产品智能体，但演讲中的比例、成效与市场定位都应保持公司自述边界。

## 5 SaaS 的分层可塑化：呈现层可生成，系统记录继续复用

### 5.1 「软件会死亡」为什么是错误的问题

主持人从 SaaS 估值和「vibe coding 将杀死软件」的流行说法切入。Rauch 不接受「软件整体死亡」这个标题；他的回答是把软件拆层：面向人的统一界面会更容易被重新生成，而系统记录、

数据、访问控制和工作流仍可能长期存在。 [00:24:07--00:25:49]

传统 SaaS 需要让同一组像素覆盖大量客户，产品自然会向「最低公分母」收敛。这里的最低公分母不是贬低产品质量，而是描述规模化产品的设计约束：过度定制会增加维护和支持成本，于是标准界面只能服务最普遍的需求。 [00:24:40--00:25:40]

## 5.2 停车管理与销售工作台：两个严格限定的案例

Rauch 讲了一个客户轶事：某位 CEO 不满意既有停车管理软件，用 v0 在一两个提示内生成替代界面，并声称节省了费用。 [00:25:49--00:26:31] 该故事没有提供数据迁移、安全、异常处理、长期维护或成本基线，因而只能说明「特定呈现层可以迅速定制」，不能证明任意生产 SaaS 都能被几句提示完整替代。

第二个例子来自 Vercel 内部销售团队。两人小组为其销售流程制作定制呈现层，但底层仍复用 Salesforce 的工作流与数据库。 [00:26:31--00:27:41] 这不是两个人重写整个 Salesforce；恰恰相反，案例的关键在于把稳定底座与可变界面分开。

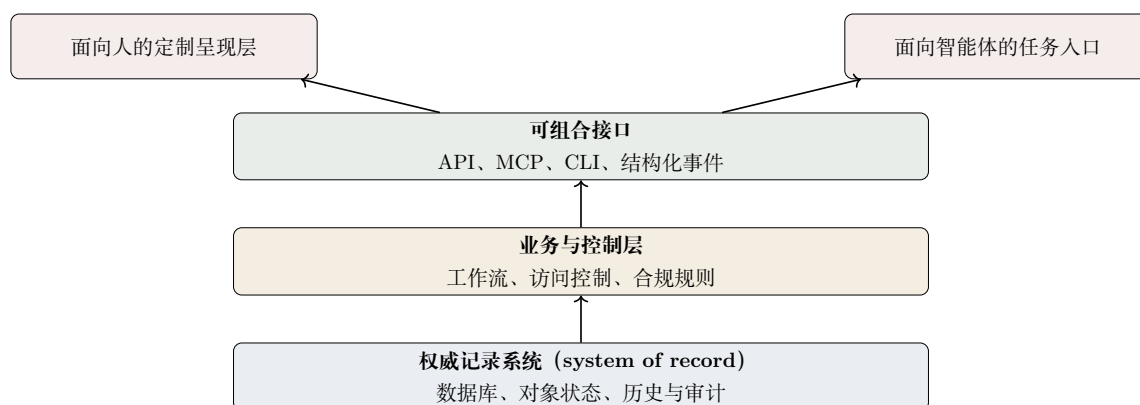


图 11: SaaS 分层：重做呈现层，同时复用记录、权限与工作流（讲义重绘；依据 00:26:31–00:28:29）

## 5.3 什么会被抛弃，什么仍值得复用

Rauch 的分层判断可以整理为三类：

1. **更易替换**：统一前端、固定报表、为最大公约数设计的交互流程；
2. **持续复用**：权威数据、访问控制、审批和业务工作流；
3. **决定能否进入新组合**：稳定 API、MCP、CLI、可理解错误与机器可读结果。

软件因此不是一次性整体死亡，而是渐进式「可塑化」。呈现层可以针对组织、角色甚至一次任务生成；底层能力若保持封闭，则难以被新界面或智能体安全复用。 [00:27:41--00:28:29]

### 局部推理在产品分层中的意义

智能体不必先理解整套 SaaS 界面，只需在清晰接口内调用数据与工作流。把依赖收拢到局部边界，既减少上下文消耗，也使人类界面与智能体入口能够分别演化。



## 5.4 本章小结

智能体没有让数据库、权限或工作流自动失去价值；它提高了呈现层的可塑性。停车和销售工作台案例都只能证明局部定制的可能性。可持续路线是把权威记录与控制机制保留为底座，通过稳定接口供不同界面复用，而不是把一个快速生成的前端误认为完整生产系统。

## 6 一次性软件与新的开发、支持闭环

### 6.1 一次客户沟通，也可以是一份软件的完整生命周期

Rauch 描述了一个售前场景：客户讲出自己的问题，团队不再只做幻灯片，而是当场或很快生成可运行、贴合客户语境的演示。若这份软件只服务一次会议，会议结束后便可丢弃；其价值不来自长期留存，而来自比文字和幻灯片更高保真的沟通。 [00:29:09--00:29:49]

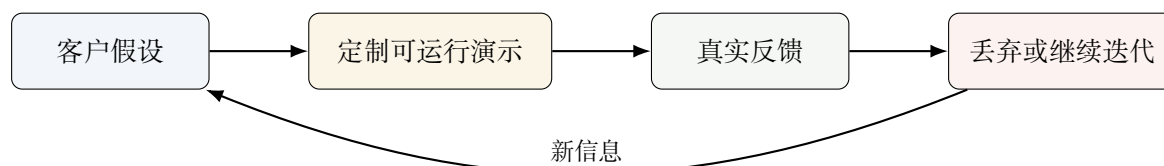


图 12: 一次性软件作为高保真沟通媒介（讲义重绘；依据 00:29:09–00:30:30）

### 6.2 「软件基本免费」只描述边际生成环节

Rauch 更喜欢说「software is basically now free」，但其紧邻语境是一次性原型、示例或演示的边际制作时间显著下降。 [00:29:40--00:30:15] 这句话不意味着模型调用、部署、数据和人员没有成本，也不意味着生成结果天然正确、安全或可维护。

#### 短命原型不能替代生产验收

一次演示失败，代价可能只是重做；基础设施失败，却可能跨客户、跨数据并持续传播。前者优化生成速度和表达保真度，后者必须优化正确性、安全、兼容性、可观测性和审慎变更。

Rauch 随即给出反例：许多基础设施软件仍然很难；有时 Vercel 会让三个智能体形成 quorum，再由多位有经验的人判断一行代码是否正确。 [00:30:30--00:31:01] 多智能体投票不是事实证明，这则轶事真正强调的是高风险软件仍需要独立证据和专家责任。

### 6.3 AI 的反身性：可生成的认知会增加生成频率

Rauch 借用 Shopify 的 Tobi 所说的「AI 的反身性」解释一个正反馈：团队知道软件可以低成本生成后，便更愿意为细小、短暂且具体的情境制作它；可运行版本更早暴露需求差异，新信息又触发下一版生成。 [00:29:56--00:30:30]

这不是正式数学模型，而是行为观察。因果链可以写成：

生成门槛下降 → 更多情境采用软件表达 → 更早获得真实反馈 → 再次生成或丢弃。



箭头是讲义归纳，不表示视频给出了可估计的系数。

## 6.4 新支持对象：客户智能体带着轨迹来到平台

过去，开发者先知道 Vercel，再读文档、调用 API，出错后向工程师说明过程。现在，有些用户甚至不知道底层平台名称：他们的智能体选择平台并完成部署，直到出错才第一次接触供应商。

[00:31:01--00:31:16]

Rauch 会请用户提供与智能体的交互 transcript，让工程师分析失败轨迹，并考虑把它转成 eval，用于修正文档、接口、模型行为或平台实现。[00:31:16--00:31:32] 与人工复述相比，轨迹能保存路径和上下文，但也可能包含密钥、个人数据与商业信息，进入支持系统前必须处理授权与脱敏。

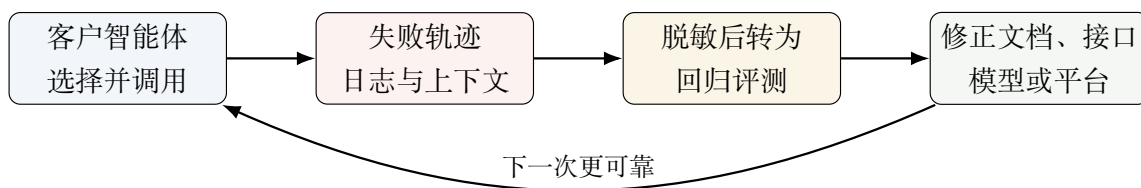


图 13: 从客户智能体失败轨迹到供应商回归评测（讲义重绘；依据 00:31:01-00:31:32）

Rauch 进一步预测支持会变成 agent-to-agent：客户智能体提交缺陷或功能请求，供应商智能体结合任务队列、token 预算、截止日期和管理偏好安排优先级。[00:31:32--00:32:09] 这是未来时态，不是一套已经完成的无人支持系统；身份、授权、可复现性和最终责任仍需人类治理。

## 6.5 本章小结

生成式软件让「只活一次会议」成为合理产品类别，也让运行演示成为高保真沟通工具。但「软件免费」只描述局部边际成本，基础设施的正确性与维护成本没有消失。与此同时，平台的新用户可能先是智能体，支持证据也从人工描述转向轨迹、日志和评测；新的闭环更短，却需要更严格的数据与责任边界。

# 7 局部推理、可组合性与全栈复用

## 7.1 高选择率不是单因果故事

主持人重新讨论图 6：所示测试中，Claude Code 对前端部署 86/86 次选择 Vercel，shadcn/ui 为 90.1% (64/71)，Vercel 为 100% (86/86)。[00:32:09--00:32:50] 画面让分母和类别可见，却仍未给出完整报告方法、模型版本或任务构造；这些数值不等于市场份额。

Rauch 的回答是多种因素共同作用。他不把结果简单归结为商业合作，也没有提供能排除合作影响的实验；更实质的解释包括多年积累的公开内容、对人和智能体都较友好的 API、技术本身的局部结构，以及访问可用性。[00:32:50--00:33:27]

### 选择结果可能由多条路径共同生成

训练或检索时可见的公开内容、接口质量、局部可组合性、品牌与商业访问都可能影响智能体的选择。没有因果分解实验时，不能只写成「产品最好」、SEO、训练数据曝光或合作中的任一单因。

## 7.2 Tailwind 的局部推理：牺牲整洁感，换取边界

Rauch 以 Tailwind 说明 local reasoning。长样式类可能违背一些开发者对代码整洁的偏好，但理解、修改或搬运一个组件时，所需信息更多地停留在组件附近，而不必先掌握全局样式系统。

[00:33:27--00:34:53]

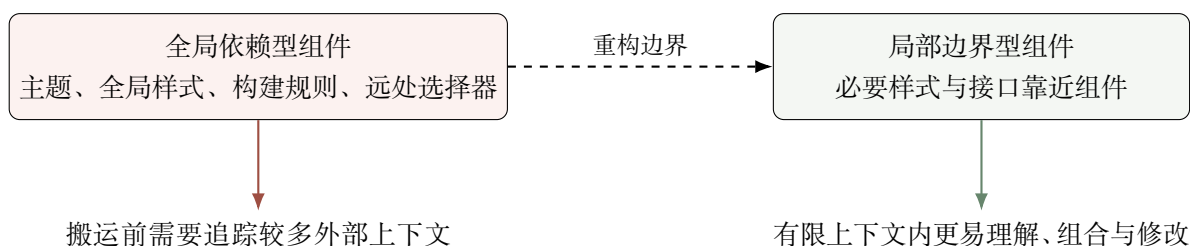


图 14: 全局依赖与局部推理的差别（讲义重绘；依据 00:33:27–00:35:36）

局部性并不保证组件在任何宿主环境都完美工作；主题、运行时与布局依然可能影响迁移。演讲表达的是设计方向：减少理解单个组件所需的远处知识。Rauch 把同一思路扩展到 React 与 Next.js，认为它原本就帮助大型人类团队扩展，如今也降低智能体的上下文负担。 [00:35:03--00:35:41]

## 7.3 有限上下文为何使可组合性成为规模化前提

Rauch 用「今天有一百万」口语化描述上下文规模，却没有说明单位、模型或有效容量，本文因此不把它当成通用规格。 [00:35:36--00:35:56] 稳定结论是：上下文始终有限。如果修改一个组件必须同时理解认证、数据库、全局样式、构建和部署，智能体会把预算消耗在远处依赖；局部边界与稳定接口则允许它在较小上下文内完成更可靠的组合。

有限上下文 → 局部边界 → headless / API 组合 → 智能体规模化能力。

公开内容让能力更容易被训练或检索，headless 形态使底层服务无需保留原界面，API 则提供稳定调用边界。Rauch 再次以 Salesforce 为例：团队仍可把它当作后端能力，却让新的应用承载定制呈现层。 [00:35:56--00:36:39]

## 7.4 全栈策略：共同底座、dogfooding 与真实负载

主持人追问 Vercel 为什么同时做 Sandbox、聊天、工作流、UI 和部署。Rauch 的回答不是「产品越多越好」，而是重要体验往往跨越关键全链路；公司也会放弃或降级没有奏效的实验。 [00:36:41--00:38:03]

Rauch 在此提出「agents 可能是最后一类软件」的强预测，用来说明他为何认为这一类别值得覆盖关键全链路。[00:37:49--00:37:53] 这是一句修辞性未来判断，不表示页面、人类界面、数据库或其他软件形态已经消失；事实上，本讲前文刚刚强调人类界面会继续丰富。

Vercel 以 dogfooding（自用验证）方式用自己的平台构建产品。AI Gateway 复用 Fluid Compute、全球网络和 CDN；Rauch 用「约 95% 的火箭发动机」比喻既有底座复用，未给出代码、能力或成本口径。[00:38:05--00:39:03] Sandbox 也复用每次部署已经使用的虚拟化原语与运营经验。[00:39:03--00:40:08]

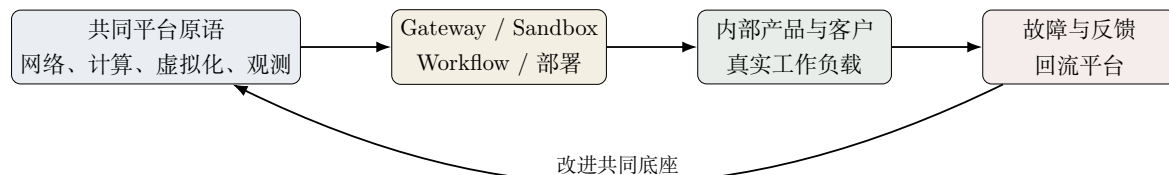


图 15: dogfooding 让共同原语承受真实负载并形成反馈（讲义重绘；依据 00:38:05–00:40:08）

Rauch 还称 Vercel 的计算量在数月内约增至三倍，自一月以来每日部署数翻倍。[00:39:38--00:39:51] 这是公司自述；基期、绝对量、截止日和统计方法未知。数字支持「真实负载正在增加」的公司叙事，不能单独证明架构优越性。

#### 全栈成立的条件

多个产品若共享成熟原语、真实负载和运营反馈，复用可以降低新能力的边际建设成本；若各层只是名称相邻，却没有共同边界与证据回流，「全栈」就只是产品清单。

## 7.5 本章小结

智能体的技术选择由多种因素共同塑造，榜单结果不能被简化为市场份额或单一因果。局部推理把必要信息收拢到组件边界，可组合接口则让有限上下文中的智能体复用底层能力。Vercel 的全栈论证建立在共同原语与 dogfooding 上；95%、3× 和部署翻倍都应保留公司自述口径。

## 8 价值归属、开放商业与「token 速度」

### 8.1 价值检验：模型能力能否抵达有用产品

主持人观察，在 2026 年的课堂语境里，许多价值似乎流向芯片、数据中心与能源等模型下层，而模型上方的应用价值较集中。[00:40:10--00:41:02] 这是一项课堂判断，本段没有给出市场统计。Rauch 则认为编码智能体可能是通向 AGI 的重要甚至主要路径；视频没有定义 AGI 或提供证明，应把它保留为强预测。[00:41:03--00:41:19]

Rauch 给出的实际检验更朴素：无论市场处于泡沫还是新阶段，关键是模型能否交付有用的服务与产品。[00:41:30--00:41:42] 模型只有接上沙箱、部署、权限、域名和治理，才会从潜在能力变成用户可以访问、命名和信任的服务。

**「最后一公里」不是装饰层**

沙箱让智能体能够工作，部署让结果可访问，域名与 DNS 让创作可命名和被发现，治理与安全限定谁能做什么。缺少任一关键环节，模型能力都可能停留在演示而非产品。

## 8.2 域名、DNS 与工程师仍然承担的责任

Rauch 转述与 Codex 团队交流时的观察：用户希望给编码智能体生成的创作命名。域名、DNS 与部署由此成为新想法进入互联网的「前门」。[00:41:42--00:43:18] 这是公司战略与二手交流，不是 OpenAI 官方用户统计；但它揭示了一个机制：只有可命名、可定位和可持续访问的对象，才能进入社会协作。

代码生成扩张没有取消工程需求。Rauch 紧接着强调智能体仍需要治理、护栏与安全，公司因此继续需要大量工程师。[00:43:18--00:43:56] 新职责不只在手写实现，还包括定义权限、建立回滚、保护共享系统、验证行为与处理异常容量。

## 8.3 承压类别与受益接口

Rauch 认为两类产品会承压：一是主要聚合静态内容的业务，他以 Stack Overflow 作为类别例子；二是把价值建立在「代码稀缺」之上的拖拽式构建器。[00:44:14--00:45:22] 这是个人市场预测，不等于所有内容平台或 low-code 产品已经失去价值。它们若拥有可信社区、权威数据、工作流或可组合接口，仍可能继续成为智能体的底层积木。

相对地，Rauch 预测智能体更偏好三种商业接口：

1. **原始信号**：结构化数据、清晰 API、机器可读错误，而不是只为人设计的营销页面；
2. **即时进入**：能够快速注册、取得有限权限并完成首次调用，而不是漫长销售流程；
3. **按量消费**：价格与实际使用更接近，能够承接机器生成的突发工作负载。

[00:45:22--00:46:21]

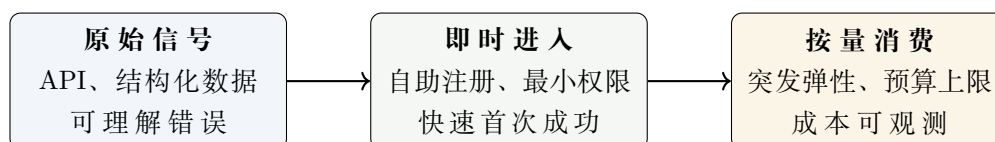


图 16: 面向智能体的商业入口：信息、访问与计费共同决定可组合性（讲义重绘；依据 00:45:22–00:46:21）

即时并不意味着绕过企业合规，按量也不等于无限信用。身份验证、合同、数据授权与预算仍然存在，只是应尽可能暴露为清晰、可编程且可审计的控制面。

## 8.4 为什么「No more rate limits」只是挑衅

Rauch 以「每分钟 100 次部署」说明静态限额的问题：数字通常把过去的人类需求假设写进代码；当调用者变成并发智能体，旧上限可能突然阻断真实业务。[00:46:21--00:47:05] 这只是教学例子，不是 Vercel 当前公开配额。

### 口号与紧邻限定 (00:47:05–00:47:40)

Rauch 提出「no more rate limits」，随即明确称它是一种 provocation。他同时列出 abuse、KYC、极端账单与系统运营健康，说明平台仍必须保护自己和其他客户。

### 不要把反对任意固定限额写成取消安全限流

目标是让容量随可信需求弹性释放，不是让未知身份无限消耗共享资源。身份、信用、预算、滥用检测、系统健康、租户公平与紧急降级都必须保留。

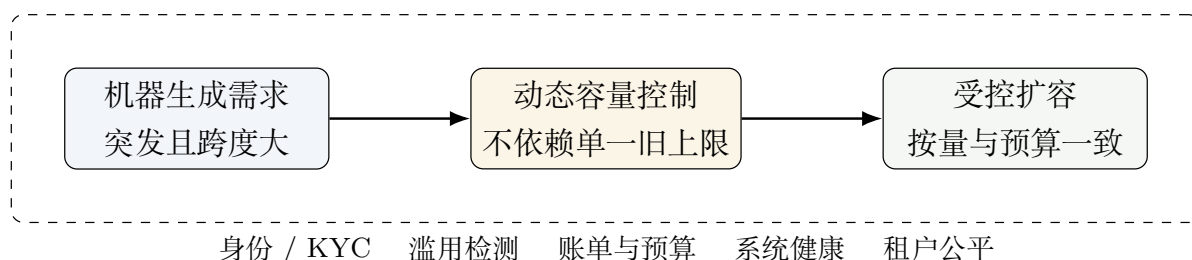


图 17: 从静态门闩转向带护栏的动态容量（讲义重绘；依据 00:46:21–00:47:40）

Rauch 又讲了一个匿名 YC 智能体平台：三个月前尚不存在，却迅速制造出超出旧配额想象的部署量。[00:47:40--00:48:05] 该案例没有公司名、绝对量或时间序列，只能说明为何机器需求会让历史容量假设失效，不能作为增长统计。

## 8.5 本章小结

价值不只聚集在芯片或模型，也可能落在把能力送到用户的最后一公里。开放接口、即时进入和按量消费更适配智能体，但合规与责任不会消失。「No more rate limits」批评的是僵化、任意的旧上限；真正目标是在身份、信用、账单、滥用与系统健康可控的条件下动态释放容量。

## 9 总结与延伸：可靠性、能源与未决问题

### 9.1 多星球兴趣背后的可靠性直觉

演讲末尾，Rauch 说如果不做 Vercel，可能会考虑太空技术。他用 multi-zone、multi-region 与 failover 描述自己的可靠性偏好，再把「单一星球」类比为更高层级的单点故障。[00:48:05--00:48:47] 这个比喻不证明移居火星是现实可行的容灾方案，也没有比较成本和风险；它只是把基础设施的冗余直觉推到极端尺度。



### 可靠性类比的正确读法

类比可以帮助记忆「避免单点故障」，却不能替代工程可行性分析。zone、region 和星球的故障模式、延迟、成本、自治能力与恢复目标完全不同。

## 9.2 「Energy goes in, intelligence goes out」

Rauch 用一句话收束全讲： *Energy goes in, intelligence goes out*. 随后他说裂变、聚变和地热方面的突破都会令他兴奋。 [00:48:47--00:49:12] 它应被理解为产业链压缩，而不是物理定律或线性转换公式。

训练和运行模型需要芯片、数据中心、供电与冷却；能源的价格、稳定性和环境代价会约束可提供的计算规模；计算经模型组织为能力，再通过网关、沙箱、部署、治理和产品转成用户效用。视频没有比较不同能源路线的成熟度、成本或安全性，也没有证明更多能源会线性地产生更多「智能」。

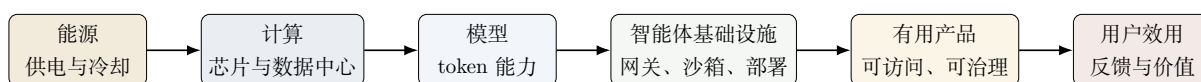


图 18: 从物理投入到用户效用的完整价值链（讲义重绘；综合 00:40:18–00:43:56 与 00:48:47–00:49:12）

## 9.3 全讲的四条教学主线

### 9.3.1 生成扩大供给，部署决定是否产生价值

自然语言和编码智能体让更多人生成软件，但代码只有部署后才能触达用户、产生行为证据并进入反馈闭环。生成越普及，沙箱、域名、DNS、观测、安全和恢复越重要。这条主线从开场的「写代码不等于交付」一直延伸到第八章的「有用产品」标准。

### 9.3.2 呈现层更可塑，底层能力通过开放接口继续复用

通用 SaaS 的统一界面可能被针对任务生成的呈现层侵蚀，但数据库、权威记录、权限和工作流仍可作为底座。要进入新组合，底层能力必须以 API、MCP、CLI 或结构化事件开放出来。局部推理把技术结构与这个商业条件连接起来。

### 9.3.3 机器需求突破静态假设，治理必须与弹性同步升级

调用者从偶尔点击的人变成并发执行的智能体后，固定配额会迅速过时。但取消任意上限不能变成取消风险控制。真正稀缺的是在身份、信用、预算、滥用和系统健康受控的前提下动态释放容量。



### 9.3.4 价值链从能源开始，以用户效用结束

能源、芯片、数据中心、模型、token 网关、沙箱、部署、域名与最终应用不是彼此孤立的类别。它们共同把物理投入变成可用服务；任何一环成为瓶颈，都可能限制整条链路。

#### 全讲最值得保留的综合结论

软件经济正在从「代码稀缺」转向「可靠运行、可信治理与开放组合能力稀缺」。这是讲义作者依据全讲结构做出的综合，不是 Rauch 的逐字原话。

### 9.4 仍待解决的问题

1. 如何定义智能体任务的可靠性，并区分「成功部署」与「真正满足用户意图」？
2. 如何让动态配额承接突发需求，同时避免错误、攻击或异常消费被平台放大？
3. 当客户智能体与供应商智能体直接交互时，身份、授权、敏感日志和最终责任归谁？
4. 开放接口如何同时满足可发现、最小权限、可审计与可撤销？
5. 一次性软件的生成成本下降后，长期维护、安全债务和软件垃圾由谁承担？
6. 能源扩大计算规模时，如何核算经济成本、碳排、用水与电网可靠性？

这些问题不是对演讲方向的否定，而是从「可能性」走向「可持续运行」所必须补齐的验证层。

### 9.5 延伸材料的正确使用方式

- Stanford Week 8 材料页列出的 Karpathy [vibe coding](#) 原帖，给出了「顺势接受模型生成、少看代码」的原始语境。它适合解释流行词的来源，不足以证明生产软件的质量。
- Karpathy 的 [Software 2.0](#) 把数据、网络架构和训练所有权重视为另一种程序表达，可帮助理解软件表示变化为何会推动新工具链；它发表于 2017 年，不是本讲的实证结果。
- 材料页还列出 [Guillermo Rauch 延伸访谈](#)。该访谈可补充「智能体既是软件产物，也是软件用户」的背景，但时间和场合不同，不能替代本讲字幕证据。

### 9.6 本章小结

结尾从多区域可靠性类比走向能源底座，完成了全讲的纵向连接：更多能源和计算可以扩大模型能力，但只有开放接口、网关、沙箱、部署、命名、治理与安全把能力可靠地送到用户手中，智能才会变成有用服务。代码生成的供给扩张不会消除工程，相反会把验证、责任、容量和物理约束推到更中心的位置。